

DSS5201 Data Visualisation

Getting started: Setup and workflow

David CHEW
Semester 2 AY25/26



Today, almost all work with some quantitative aspects involves coding.

- An **essential skill** across a wide range of domains.
 - Widely used across industry, academia, and the public sector.
- Many of the tools and packages that have been developed will help you do better work more productively.

Learning basic programming concepts in *any* programming language is useful for almost any other language.

Python and R

Python and R are powerful tools in data analysis.

	Python 🐍	R 📊
Pros	Larger global user base	Very active community
	Easy to collaborate with others	Easy to use packages
	Best for machine learning and production deployment	Best for statistical analysis and data visualization
Cons	Less specialized for statistical analysis	Limited scalability
	Choosing libraries and IDEs can be overwhelming	May be harder to integrate with production pipelines
Data manipulation	<code>pandas</code>	Tidyverse (<code>dplyr</code> , <code>tidyr</code> , ...)
Visualization	<code>matplotlib</code> , <code>seaborn</code> , ...	<code>ggplot2</code>

Prerequisites



In this course we will use Python.

- One of the most popular programming language in the world.
- A general purpose language that perform a wide range of tasks.
- Also, one of the easiest to learn.

Setting up

Install essential software and extensions

- Install Python
- Install VS Code
- Install extensions in VS Code

Package management

- Create virtual environment in Python
- Activate the virtual environment
- Install necessary packages in the virtual environment

Test the setup with the sample notebook.

A few concepts

- **Python** is a programming language with its own rules and syntax.
- An **Operating system** manages a computer's hardware and provides a platform for programs to run.
- A **Python interpreter** (e.g., Python 3.14.2) is a program that reads and executes Python code.
- A **Code editor** (like VS Code) is a place we write and run code.
- **Packages** are add-on modules that extend Python's functionality.

Python, VS Code, and Python packages

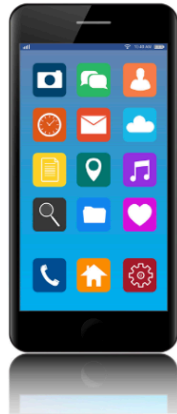
Python interpreter: Engine



VS Code: Driver's seat



Python interpreter: A new phone



Python Packages: Apps we can download



Dedicated folder for DSS5201

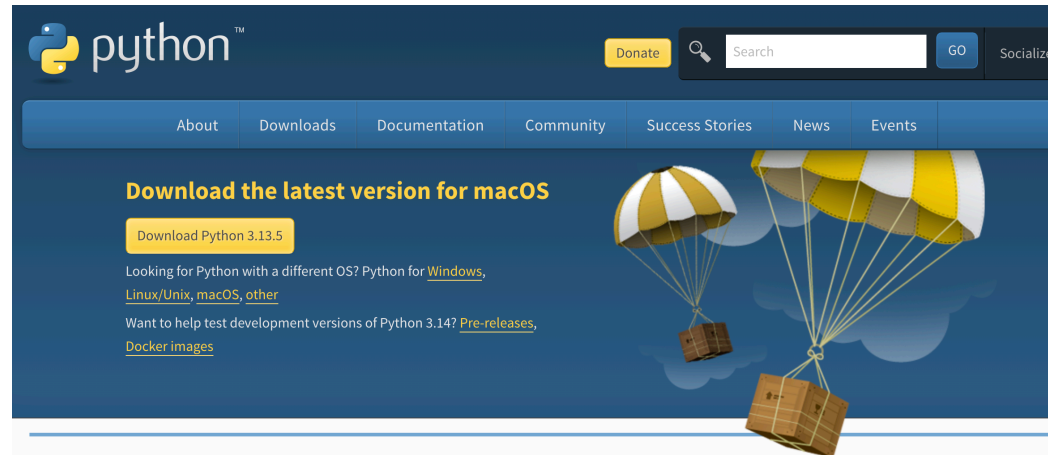
Organize your files for DSS5201 using the following structure:

```
1 DSS5201
2   |--- data
3   |--- src
```

Throughout the semester, we will store our

- source files (`.py`, `.ipynb`) in the `src` sub-folder;
- data files in the `data` sub-folder.

Installing Python



We first install a Python interpreter.

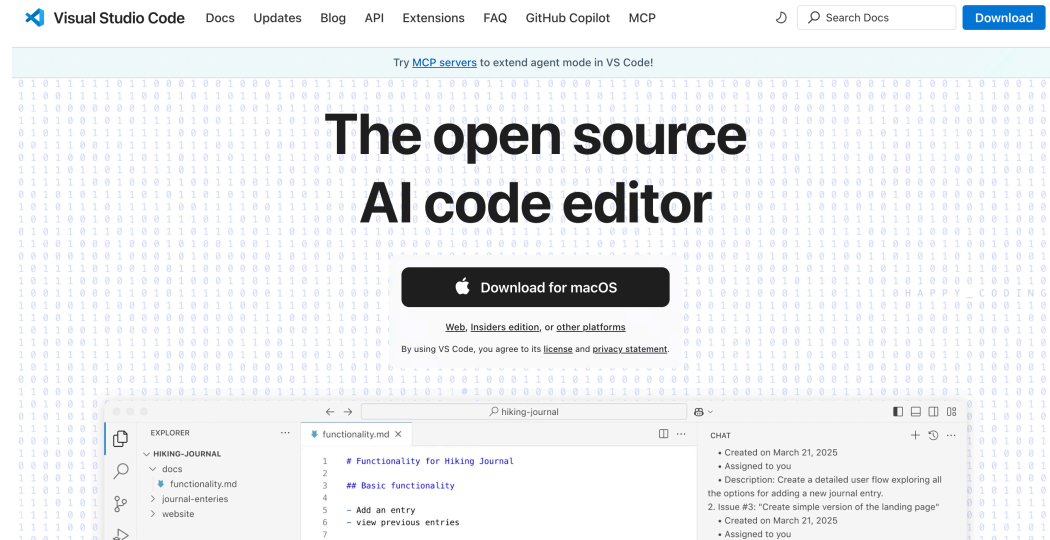
- Navigate to the official [Python Downloads page](#) to obtain the appropriate installer for your operating system.
- For this course, ensure you are using at least **Python 3.12**.

Installing Python (Windows)

- **Windows** users: Select “Add python.exe to PATH” during installation.
- This allows us to run Python from the Terminal without extra setup.



Installing VS Code



Microsoft's **VS Code** is a free, lightweight code editor that makes coding easier.

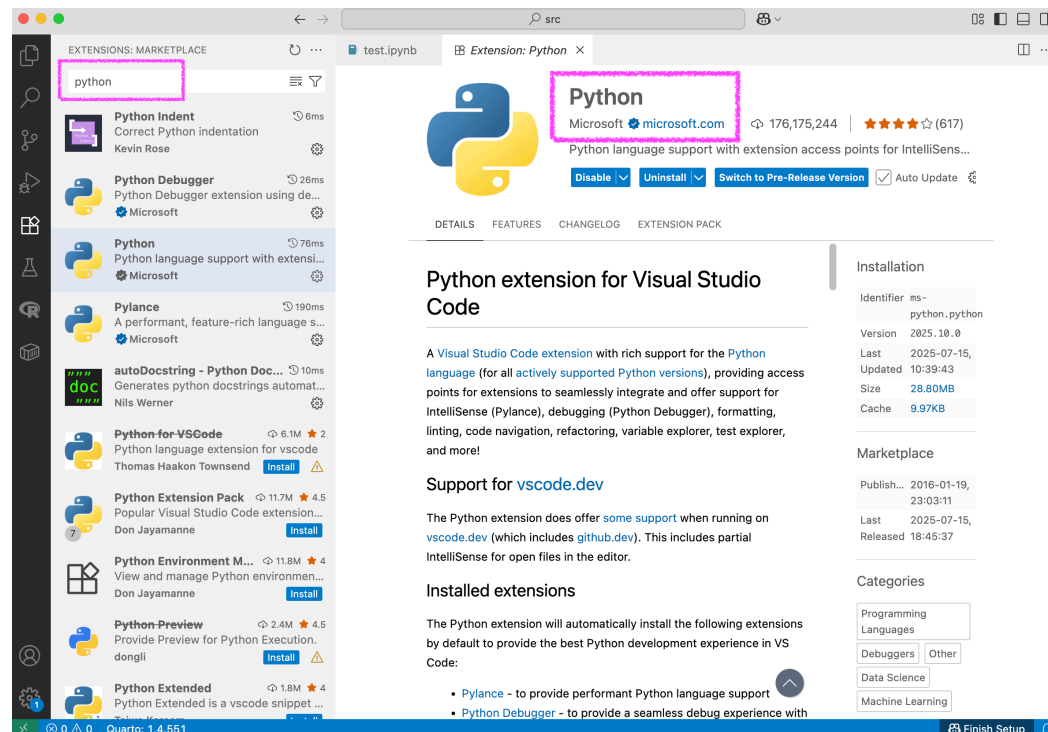
- Beginner-friendly.
- Run code interactively in Jupyter notebooks.
- Check code for basic errors.
- Highly customizable with extensions.

Download and install VS Code from the **official website**.

VS Code Extensions

Click on the activity bar to open the Extension panel.

- Search for and install the following extensions:
 - Python, Jupyter, Jupyter Notebook Renderers
- After that, restart VS Code.

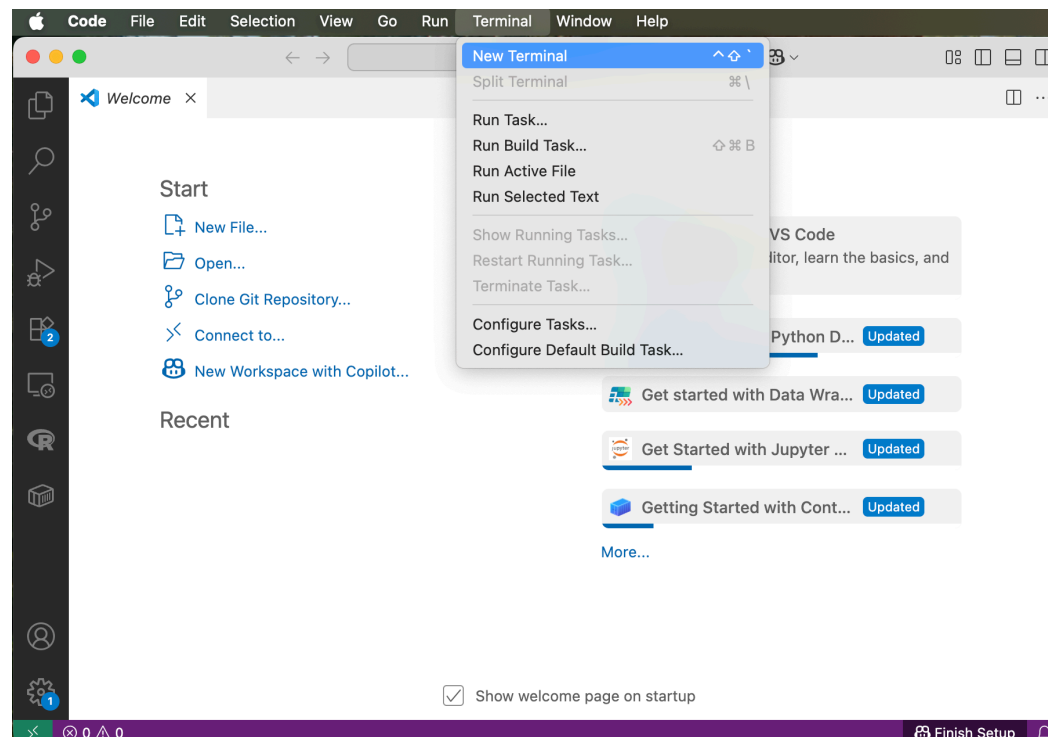


VS Code Terminal

Restart VS Code.

Open the **Terminal** (**Terminal** > **New Terminal**) in VS Code.

- The terminal will then appear at the bottom of the VS Code window. We can type commands there, just like in a regular **Command Prompt** (Windows) or **Terminal** (macOS).



Verify the installation

Type and run the following in the **Terminal**:

```
1 python3 --version
```

You should see the version of Python currently installed.

Note: the following commands should also do the same thing for Windows:

```
1 py --version  
2 python --version
```

A word about “system Python”

Python is an essential part of many Linux distributions.

- macOS and Linux may include a pre-installed system Python.
- The version is usually outdated (e.g., Python 2.7) and is missing out updates in newer releases.
- Not intended for use in this course.

For our work, we will use the latest stable release of Python and manage packages through a local virtual environment.

Setting up

Install essential software and extensions

- Install Python
- Install VS Code
- Install extensions in VS Code

Package management

- Create virtual environment in Python
- Activate the virtual environment
- Install necessary packages in the virtual environment

Test the setup with the sample notebook.

Package management

A **package** (library) is a collection of function, data, and documentation that extends the capabilities of Python.

Some **standard libraries** are built-in with Python:

- **os** to navigate folders and manage files.
- **sys** to access system-specific parameters and functions.
- **datetime** to handle dates and times.

Additional Packages

The Python community also develops many powerful packages, such as

- `numpy` for numerical computing
- `matplotlib` for basic visualization, and
- `pandas` for data manipulation.

We can load packages into our workspace with the `import` statement.

Package versions

Each package has its own version (e.g., `pandas==1.5.3` vs `pandas==2.3.1`).

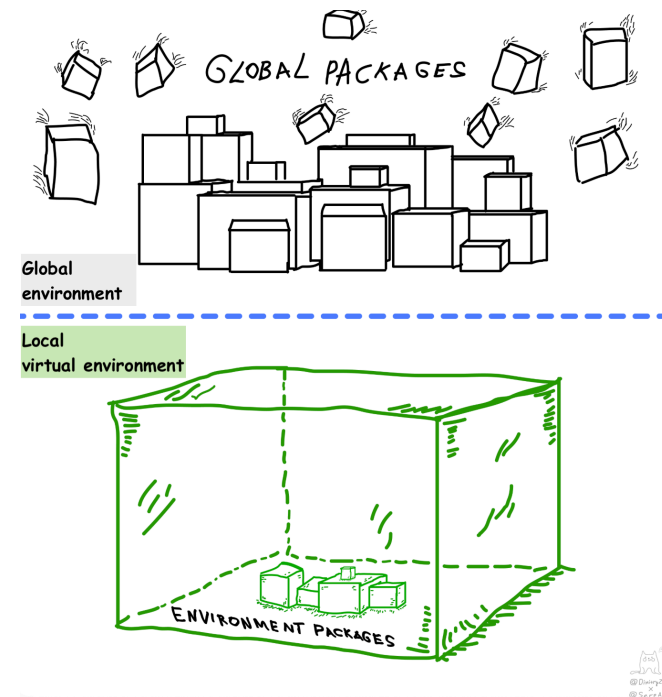
- Different versions can behave differently – a function that works in one version may be changed in another version.
- Using a different version can
 - break your code
 - produce unexpected results
 - conflict with other packages

To prevent these issues, the best practice is to create an **isolated environment** where we control exactly which package versions are installed and used.

What is an environment?

A **Python environment** is a context in which we run Python code.

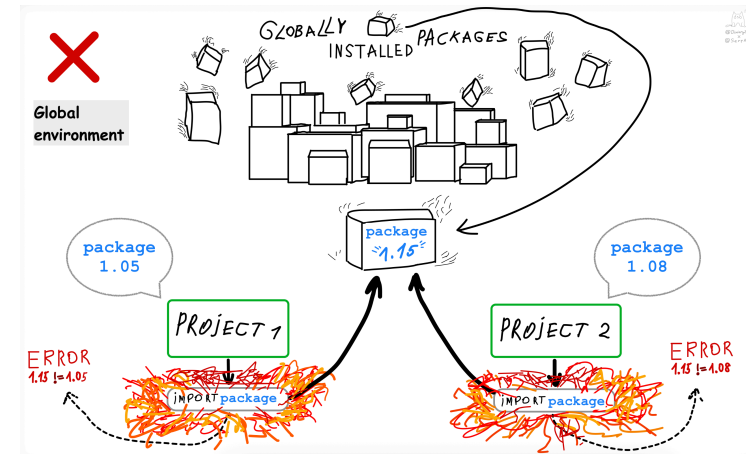
- The global environment has a single version of each installed package.
- A local virtual environment allows different projects to use different versions of Python and packages.
- As a best practice, we will create a **DSS5201-specific virtual environment** to keep our setup clean and avoid package conflicts.



Global environment

The **Global environment** has a single version of each package installed.

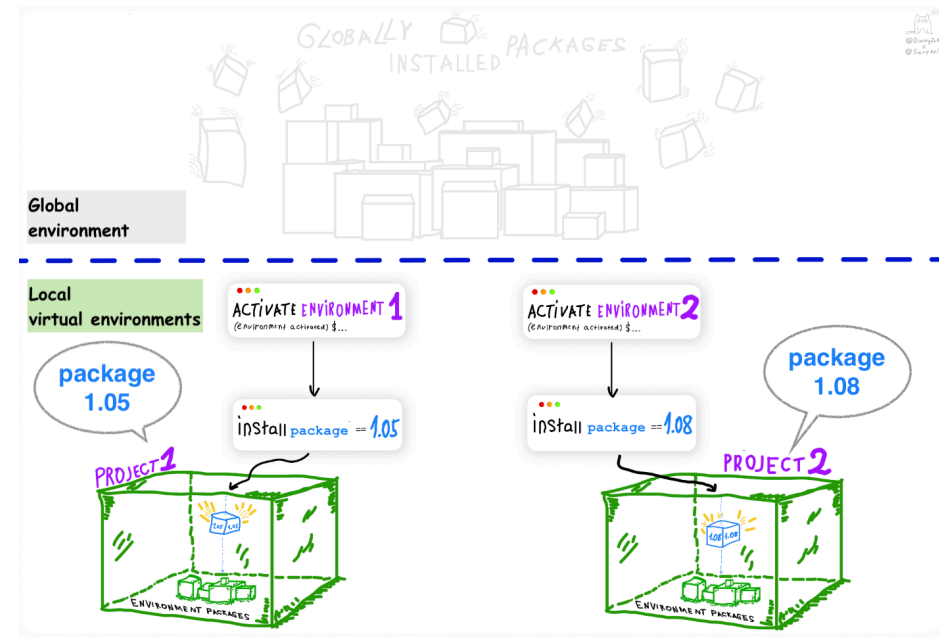
- Only a single version of package can be installed with a given Python interpreter.
- If a project expects a different (newer or older) version of the same package, it might break or show errors.
- We want to avoid installing packages globally when working on multiple projects.



Virtual environment

Virtual environments allow multiple versions of Python and packages to coexist with each other.

- Each environment is isolated.
- **Full control** on the version of Python used, the packages installed, and when each gets upgraded.
- The setup is clean and prevents conflicts.



Adapted from Dmitriy Zub and Sonika Arora.

Always use a virtual environment

`venv` is Python's built-in tool to create isolated virtual environments.

- Allows us have a stable, reproducible, and portable environment.
- Ideal to create one for each project (or class).

To ensure a consistent coding environment, we will create a DSS5201-specific environment for this course.

- Keeps our setup clean and reproducible.
- ... especially important for collaboration and assessments.

One-time setup

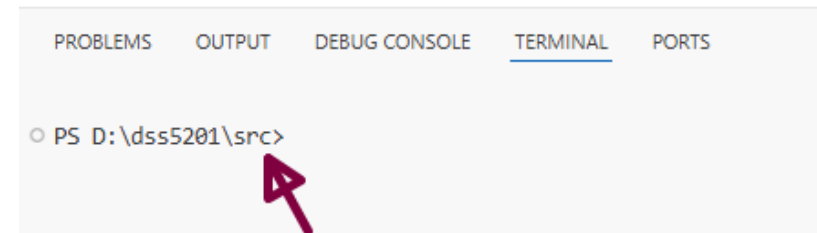
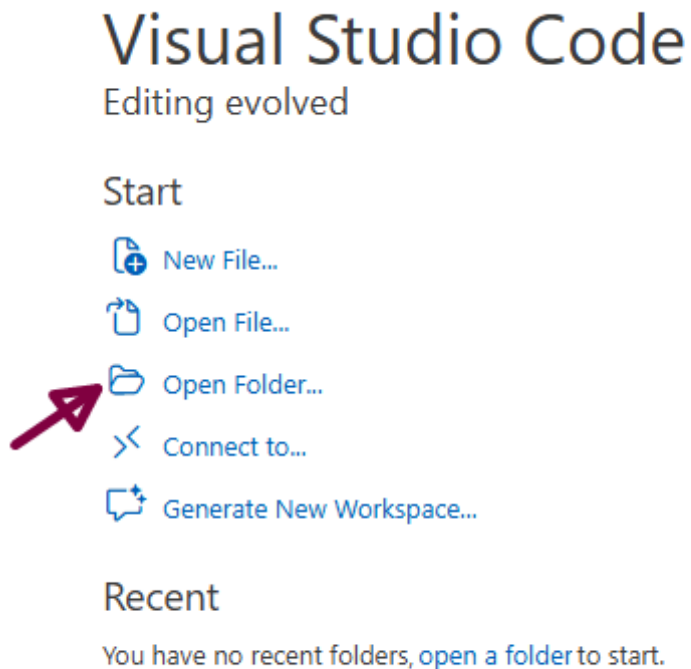
We will now create a virtual environment for DSS5201.

- Recall our standard folder structure:

```
1 DSS5201
2   |--- data
3   |--- src
```


One-time setup

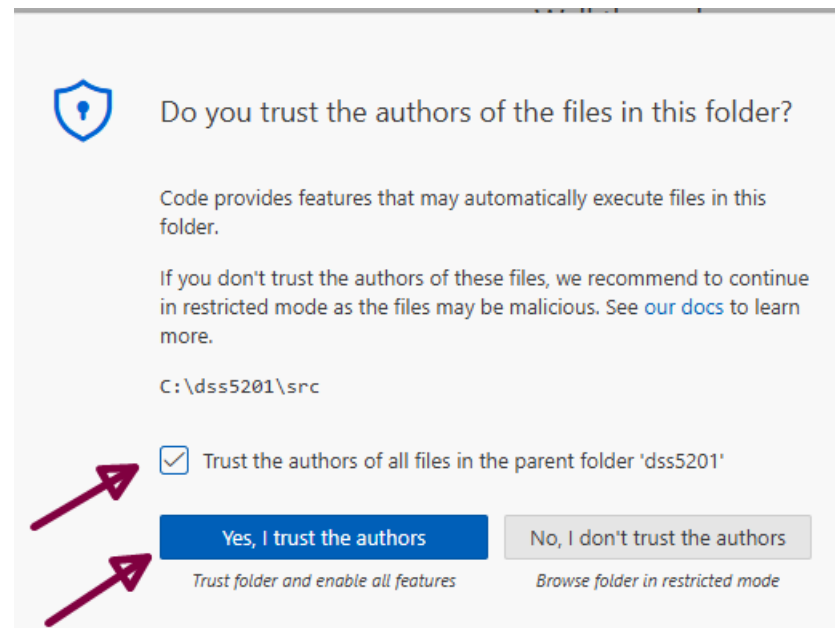
- Launch VS Code and navigate to the `src` folder.
- This ensures the terminal commands we'll use later run in the correct location.



One-time setup

When we open a folder in VS Code, we might see this prompt.

- Check “Trust the authors of all files in the parent folder xxx”.
- Then click “Yes, I trust the authors”.



One-time setup

In VS Code's integrated Terminal, type and run the following command:

```
1 python3 -m venv venv-dss5201
```

- This creates a new folder named `venv-dss5201` in `src`.
- It contains all the files for the virtual environment.

Activating the environment

Before running or installing anything, we need to activate the environment.

This tells VS Code to use the environment's Python interpreter and installed packages.

- Windows

```
1  venv-dss5201\Scripts\activate
```

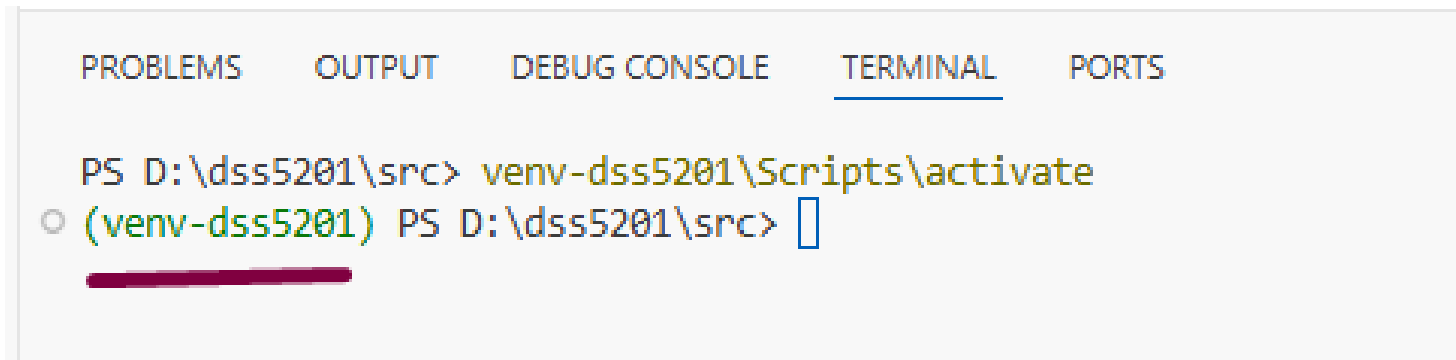
- macOS

```
1  source venv-dss5201/bin/activate
```

Activating the environment

Once activated, we will see the environment name (`venv-dss5201`) appear in our Terminal.

- This confirms that the virtual environment is active.



The screenshot shows a VS Code interface with a terminal window open. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is selected and underlined), and PORTS. The terminal content shows a PowerShell prompt at 'D:\dss5201\src' where the command 'venv-dss5201\Scripts\activate' has been entered. The prompt has changed to '(venv-dss5201) PS D:\dss5201\src>', indicating the virtual environment is active. A red horizontal bar is visible below the prompt.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\dss5201\src> venv-dss5201\Scripts\activate
(venv-dss5201) PS D:\dss5201\src> 
```

Installing packages

`pip` is Python's built-in package manager.

- Once the environment is active, we can install packages with `pip install`.

```
1 pip install numpy
```

- To check the current setup and the installed packages:

```
1 pip list
```

Installing packages

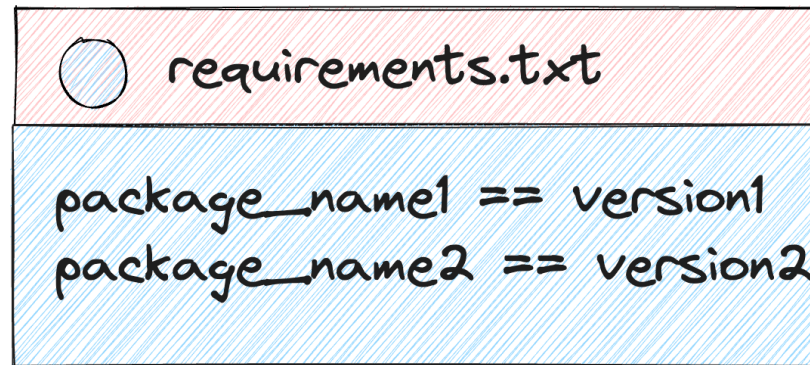
We can install a single package, or install multiple packages at once.

- Download `requirements.txt` on Canvas to the `src` folder.
- Then run the following command:

```
1 pip install -r requirements.txt
```

- You may see a message suggesting to upgrade `pip`. It is safe to upgrade.

```
1 pip install --upgrade pip
```



The `requirements.txt` lists a set of Python packages we need.

- Can also specify versions if needed.
- It allows team mates (and your future self!) to recreate the same environment.
- Reproducibility: Provides a consistent environment and makes collaboration easier.

Looking into the requirements

The `requirements.txt` includes the key packages we'll work with in the first few weeks.

```
1  # core
2  jupyter
3  nbconvert
4  ipykernel
5  # data sets
6  pydataset
7  nycflights13
8  # data manipulation
9  numpy
10 pandas
11 # data import
12 openpyxl
13 xlrd
14 # data visualization
15 matplotlib
16 seaborn
```

Creating a requirements.txt

As the semester progresses, we might install additional packages in the local environment.

- To extract the packages, we use the `freeze` command:

```
1 pip freeze > requirements.txt
```

- This generates a list of installed packages with exact versions.
- ... including the dependencies of these packages.
- Can be used to re-create the same environment on another machine.

Removing a package

Once a package is installed, it becomes available only in this virtual environment.

It will stay unless we explicitly remove it with `pip uninstall`.

Deactivating the virtual environment

We can run `deactivate` to exit the virtual environment.

```
1 deactivate
```

- This avoids accidentally installing packages in the wrong environment.
- ... especially useful when working across multiple project environments.

Managing environments

Virtual environments are isolated for reproducibility and package control.

The trade-off is disk usage.

Best practice:

1. Reuse environment for similar projects. Use `venv-dss5201` for all DSS5201-related work.
2. Clean up old or unused environments periodically.

- macOS

```
1 rm -rf name-of-old-venv
```

- Windows

```
1 rmdir -r name-of-old-venv
```

Setting up

Install essential software and extensions

- Install Python
- Install VS Code
- Install extensions in VS Code

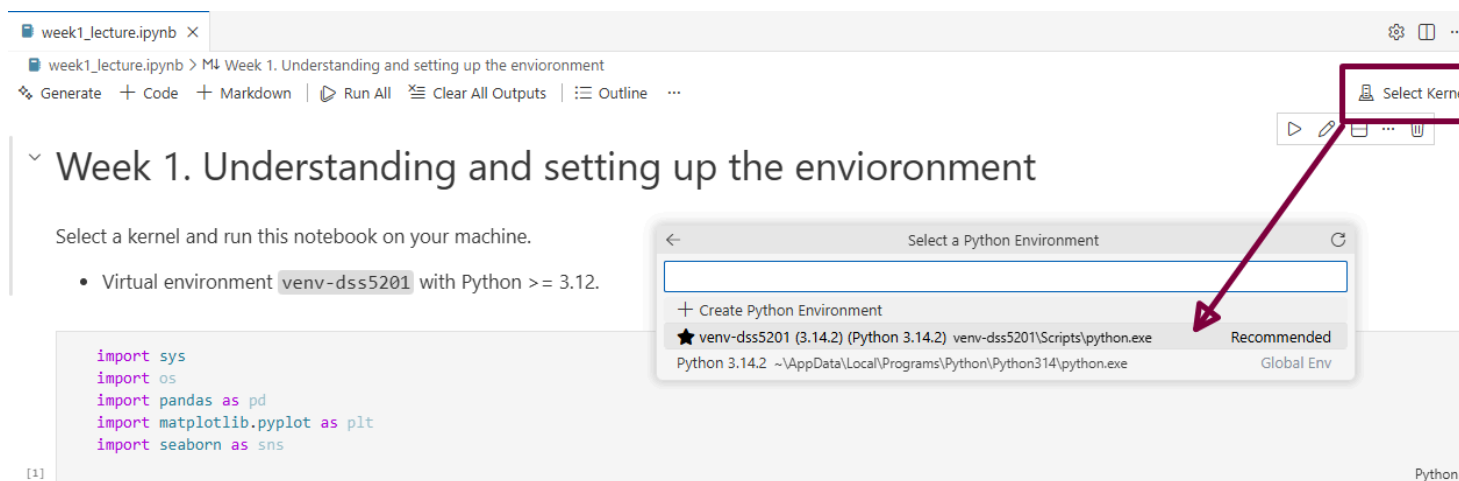
Package management

- Create virtual environment in Python
- Activate the virtual environment
- Install necessary packages in the virtual environment

Test the setup with the sample notebook.

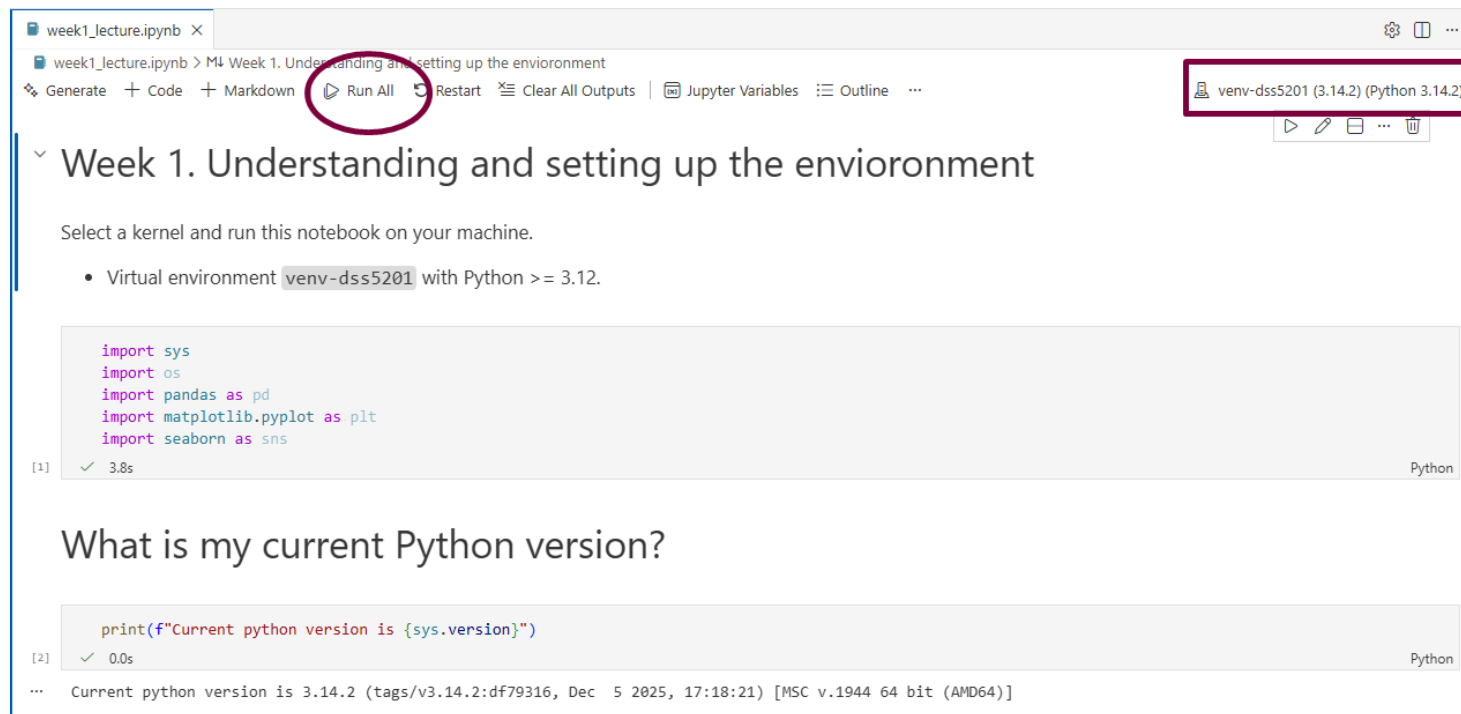
Let's now test the setup

- Download `week1_lecture.ipynb` from Canvas to your `src` folder.
- Open it in VS Code.
- Click Select Kernel and choose `venv-dss5201` as the Python environment.



Let's now test the setup

- Click Run All to run the code cells.



The screenshot shows a Jupyter Notebook titled 'week1_lecture.ipynb'. The interface includes a top bar with tabs for 'Generate', 'Code', 'Markdown', 'Run All', 'Restart', 'Clear All Outputs', 'Jupyter Variables', and 'Outline'. The 'Run All' button is circled in red. In the top right corner, the kernel name 'venv-dss5201 (3.14.2) (Python 3.14.2)' is highlighted with a red box. The notebook content starts with a section titled 'Week 1. Understanding and setting up the environment'. Below this, there is a text prompt: 'Select a kernel and run this notebook on your machine.' followed by a bullet point: 'Virtual environment venv-dss5201 with Python >= 3.12.' The first code cell contains the following imports:

```
import sys
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

 This cell is executed successfully, showing a green checkmark and a runtime time of 3.8s. The second code cell contains the command:

```
print(f"Current python version is {sys.version}")
```

 This cell is also executed successfully, showing a green checkmark and a runtime time of 0.0s. The output of the second cell is: 'Current python version is 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]'.

Export a notebook to HTML

For projects and exams, you will be asked to submit **both** the notebook and its HTML output.

To export notebook to HTML:

- Look for the Export button and select Export to HTML.



Export a notebook to HTML

- Alternatively, you can export your notebook with this command in the terminal:

```
1 jupyter nbconvert --to html --execute week1_lecture.ipynb
```

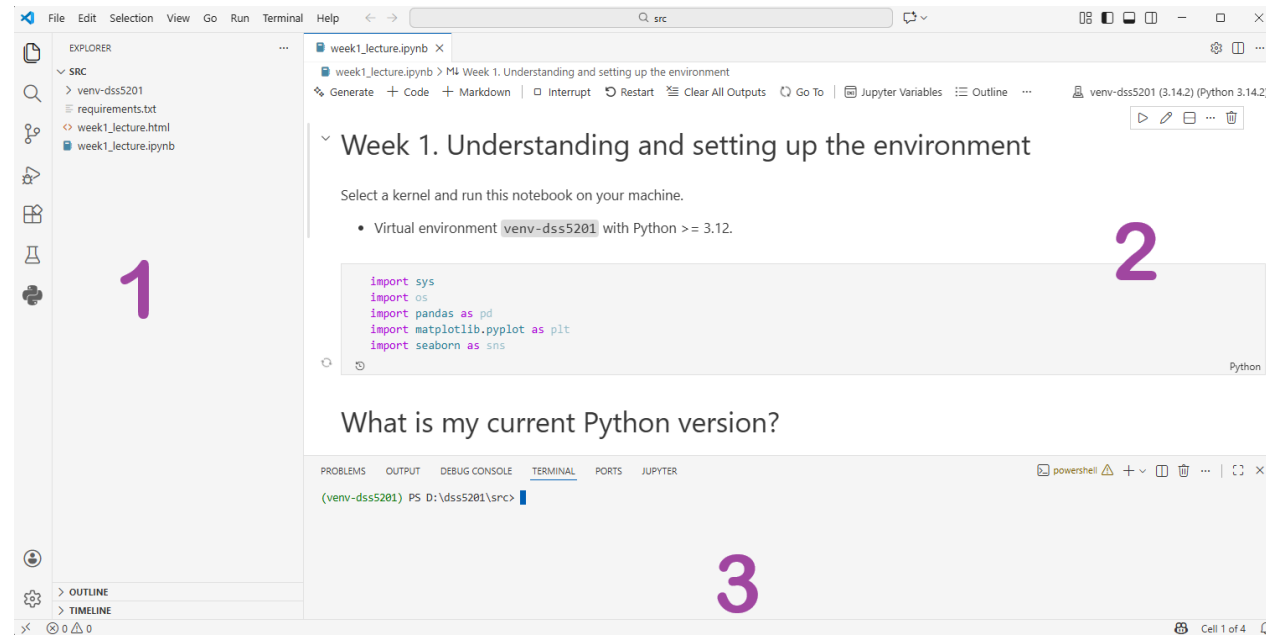
Folder structure, again

By now, your `DSS5201` folder should look like this:

```
1 DSS5201
2 |-- data          # Data sets
3 |-- src           # Scripts and notebooks
4     |-- venv-dss5201 # Virtual environment folder
5     |-- requirements.txt # List of required packages
6     |-- week1_lecture.html # Rendered HTML output
7     |-- week1_lecture.ipynb # Sample test notebook
```

- We can create additional folders (e.g., for lecture notes, projects) inside `DSS5201`.
- Keep the `data` and `src` structure unchanged – this ensures file paths in our code work as expected.
- We will go over working directories and file paths in the subsequent weeks.

The VS Code interface



1. The **Explorer**, showing the content of the folder that's currently open.
2. VS Code **Editor**, showing a Python Notebook.
3. The **Terminal**, a place where you can type in commands and your computer will execute.

Jupyter Notebook

Throughout the semester, we will use **Jupyter Notebooks**.

- Beginner-friendly and easy to interact with code.
- We can run the code cell(s) and view the output directly below the code.

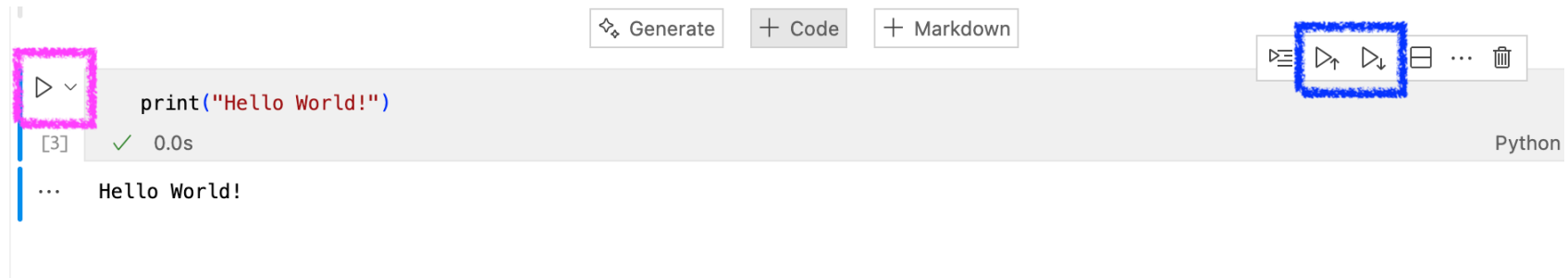
Try it out:

- Create a new code cell.
- Write a simple print expression to display “Hello World!”.

```
1 print("Hello World!")
```



Jupyter Notebook

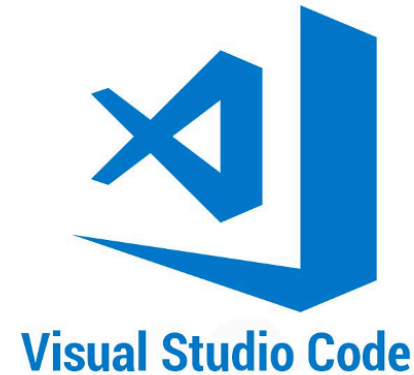


Add a code cell and write a print expression to display “Hello World!”.

Three types of run buttons:

- Run current cell.
- Run above cells.
- Run current cell and below.

Learning by doing



Lectures for this course will be highly **interactive** and requires **lots of learning-by-doing**.

- We recommend that you create your own notebook to following along on your own laptop.
- As the lecture progresses, type out the code examples from the slides.
- Run the code to see what it does.
- Don't just copy. Tweak the code and experiment with changes to understand how it works!

Error messages

Making mistakes is a natural part of programming. Everyone encounters errors!

When an error occurs, Python displays a message with

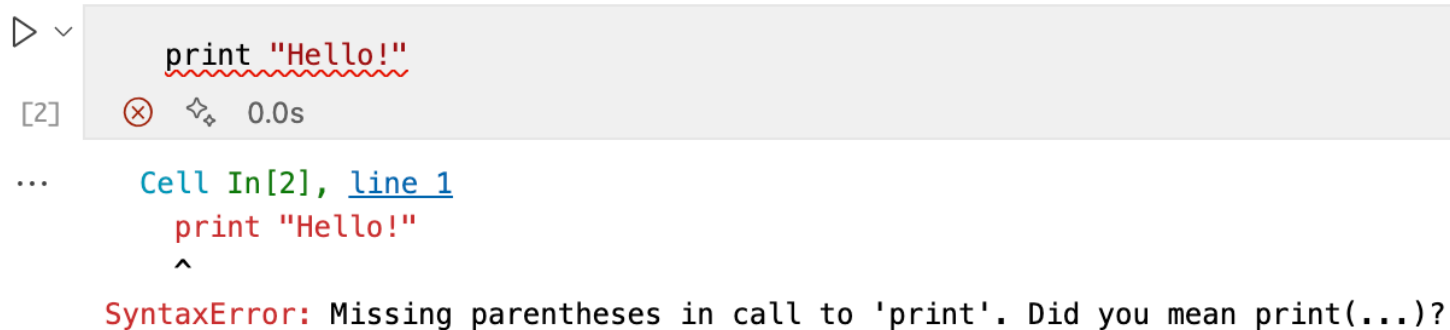
- The cell and line number of the error.
- The type of error (e.g., `SyntaxError`).
- Additional details to help us understand what went wrong.

Learning to read and interpret error messages is an important skill.

Syntax error

Typing a print statement without parentheses is a syntax error.

- The caret symbol (^) shows where Python finds the error.



The screenshot shows a Jupyter Notebook interface. At the top, there is a run button (a triangle) and a dropdown arrow. Below it, a code cell is displayed with the text `print "Hello!"`. The word `print` is underlined with a red wavy line. To the left of the code, there is a prompt `[2]`, a red circle with a white 'x' icon, a star icon, and the text `0.0s`. Below the code cell, there is an ellipsis `...` followed by the text `Cell In[2], line 1` and `print "Hello!"`. Underneath the code, a red caret symbol (^) points to the first character of the line. Below the caret, the error message `SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?` is displayed in red text.

- Python expects `print()` instead of just `print`.

Mistake	Error message	Explanation
<code>print("Have a nice day!")</code>	SyntaxError: unexpected EOF while parsing	The closing parenthesis is missing. Python is surprised to reach the end of file (EOF) before this line is complete.
<code>word = input("Type a word: ")</code>	SyntaxError: EOL while scanning string literal	The closing quote marks are missing. As a result, the string does not terminate before the end of line (EOL).
<code>print("You typed:", wurd)</code>	NameError: name 'wurd' is not defined	The spelling of word is incorrect. The programmer accidentally typed the wrong key.
<code>prints("You typed:", word)</code>	NameError: name 'prints' is not defined	The spelling of <code>print</code> is incorrect. The programmer accidentally typed an extra letter.
<code>print("Hello")</code>	IndentationError: unexpected indent	The programmer accidentally typed a space at the start of the line.
<code>print("Goodbye")</code>	IndentationError: unexpected indent	The programmer accidentally pressed the Tab key at the start of the line.

Additional resources on setup

1. `venv` - Creation of Virtual Environments.

<https://docs.python.org/3/library/venv.html>

2. Python Environments in VS Code.

<https://code.visualstudio.com/docs/python/environments>

3. Jupyter Notebooks in VS Code.

<https://code.visualstudio.com/docs/datascience/jupyter-notebooks>